UNIVERSITY OF TRIESTE

Engineering and Architecture Department

Computer Engineering

Control of Cyber-Physical Systems

Report of: Lateral Vehicle Optimization Using Constrained Linear Time-Varying MPC

Author:
Michelangelo Pozza
ID: IN2300012

Teacher:
Prof.
Davide Martino Raimondo

Academic Year 2025/2026

Contents

Contents **2**

1 Trajectory Generation and Velocity Profile **4**
1.1 Curves and Parametrizations 4
1.2 Formal Definition 4
1.3 Regular Curves 4
1.4 Intrinsic Parametrization 5
1.5 Arc-Length Parametrization 5
1.6 Curvature 6
1.7 Arc-Length Parametrization - Implementation 7
1.8 Velocity Profile 8

2 Vehicle Prediction Model **10**
2.1 Non-Linear System's Equations 10
2.2 Non-Linear System - Implementation 10
2.3 Linear System Model 11
2.4 Discrete-Time Linear System Model 13
2.5 Linear System Model - Implementation 13

3 System Constraints **15**
3.1 System Constraints - Implementation 16

4 Cost Function **17**
4.1 Cost Function - Implementation 18

5 Constrained Optimal Control Problem **19**
5.1 Implementation in MATLAB 21
5.2 Slack Variables 21
5.3 Slack Variables and Constraints - Implementation 25

6 Results **28**
6.1 Paper's results 28
6.2 My results 28
6.2.1 Straight road 28
6.2.2 Sine() road 28
6.2.3 Roundabout 29
6.2.4 Parking scenario 30
6.2.5 Lemniscate 30
6.2.6 Simple Curve 31

Introduction

In recent years, autonomous driving has advanced rapidly. Various companies are implementing different strategies to make vehicles more intelligent and enhance passenger comfort. A branch of the research in this field focuses on trajectory generation algorithms that can cope with complex traffic featuring dynamic, time-critical street scenarios, such as merging into fast traffic flow, to pass opposing traffic, or to avoid other moving vehicles.
In the literature, different methods have been explored to address these problems: starting from simple heuristics in combination with path-planning strategies designed for isolated dynamic traffic scenarios. These approaches reach their limits when applied to real scenarios. Potential field methods were also applied with some good results but they had problems when dealing with constraints. Research has shifted toward the optimization algorithms where we can distinguish three different methods:

- *Dynamic Programming*: this method can be applied to a broad range of universal system models. Different terms can be easily integrated into the cost function, as well as constraints. Moreover, non-convex optimization problems can be solved globally. Unfortunately we have to deal with the *curse of dimensionality*: due to the high number of calculations of this approach cannot be applied to complex, high-order systems. The authors decided to use this technique just for the parking scenario.

- *Indirect Methods*: this method can be applied only to local optimization problems and they show great potential with respect to trajectory generation in some special cases. For simple system models solution to the optimization problem can be found efficiently and an analytical solution can be pre-computed offline. However, considering system constraints explicitly within the optimization is extremely difficult. Also, initial conditions for the adjoint variales must be provided at the beginning, making indirect methods unflexible for online application.

- *Direct Methods* convert the optimal control problem into nonlinear programming that can be solved numerically. They are very adequate to find local solutions for nonlinear problems in real-time. The problem of these methods consists on the high dependancy of the solution on the initial conditions of the numerical solver, which are not easy to find.

The authors decided to use the formulation of a linear-quadratic optimal control problem for trajectory generation. The fact that the problem is simply to solve by a general quadratic solver and does not need any particular tuning makes the choice of the LMPC highly favorable for real-time trajectory generation with high replanning frequencies.
The authors of the paper implement the final result on a real vehicle and tested it. Also, during simulation they use real trajectories and obstacles positions without describing any general methodology to generate these nominal trajectories.

1 Trajectory Generation and Velocity Profile

The chosen paper presents a significant challenge that emerged during implementation: the reference curve Γ that the vehicle has to follow by optimizing the lateral distance is not provided in any case. In order to implement the paper we need to find a way to generate these trajectories.

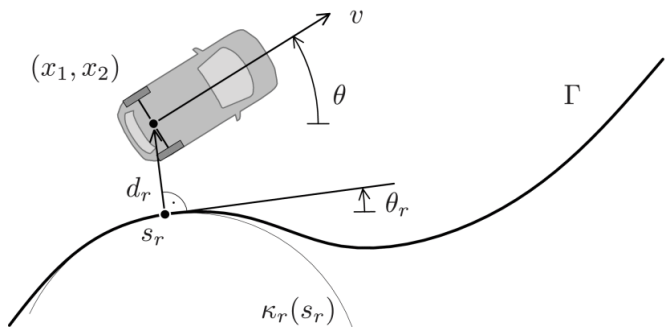

Figure 1.1: Vehicle model with the given curve Γ

1.1 Curves and Parametrizations

Intuitively, a *curve* in n-dimensional space can be thought of as the trajectory of a point moving through space over a certain interval of time. However, in differential geometry, it is essential to distinguish between the geometric object itself (the path) and the mathematical function used to describe it.

1.2 Formal Definition

A *parametrized curve* is defined as a continuous map $\Gamma : I \to \mathbb{R}^n$, where $I = [a, b]$ is an interval of the real line. The set of points in the image $\mathcal{C} = \Gamma(I)$ is called the *support* or the *trace* of the curve.
While the support represents the "trail" left in space, the function $\Gamma(t)$ specifies *how* this trail is traversed as the parameter t varies. It is important to note that the same support can be described by infinitely many different parametrizations. For instance, if we consider t as time, different functions $\Gamma(t)$ can represent the same path covered at different speeds or even in opposite directions.

1.3 Regular Curves

To study the geometric properties of a curve (such as its curvature), we generally require the curve to be "smooth" enough. We say that a curve $\Gamma(t)$ is *regular* if:

1. $\Gamma(t)$ is C^1;
2. $\Gamma'(t) \neq 0$ for all $t \in I$.

The condition $\Gamma'(t) \neq 0$ ensures that the moving point never comes to a momentary halt. Geometrically, this guarantees that a *tangent vector* is well-defined at every

point, preventing the existence of sharp corners or "cusps" that would make the calculation of curvature impossible.

1.4 Intrinsic Parametrization

Since the physical shape of a curve does not depend on the speed at which we choose to traverse it, it is often useful to employ a "natural" parametrization that depends only on the geometry of the trace. This is the *arc-length parametrization*. In this setting, the parameter no longer represents a generic time t, but rather the actual distance traveled along the curve, providing an intrinsic way to analyze its properties.

1.5 Arc-Length Parametrization

Given a general differentiable curve $\Gamma : I = [a, b] \to \mathbb{R}^n$, the arc-length parameter $s_r(t) : I \to \mathbb{R}$ is defined by

$$s_r(t) = \int_a^t ||\Gamma'(u)||du \tag{1}$$

where $t \in I$.
To visualize the difference between a general parametrization in t and an arc-length parametrization in s we can look at the example in image 1.2.

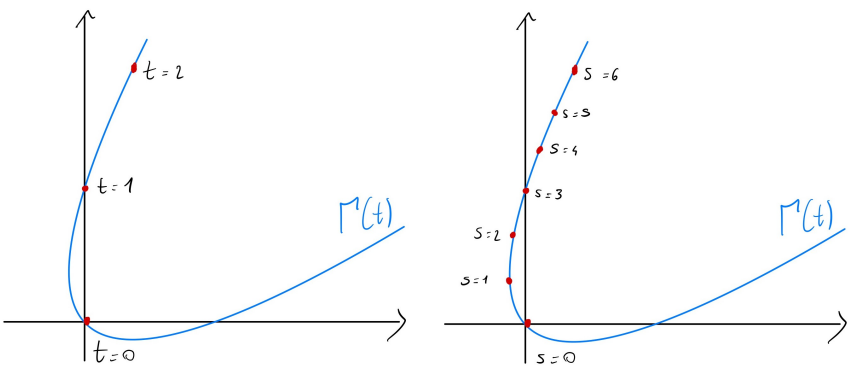

Figure 1.2: Difference of parametrizations

There exits a direct relationship between any parametrization and the arc-length parametrization. Let's see an example in an analytical way. Consider the curve described by

$$\Gamma(t) = \begin{bmatrix} 3t - 1 \\ 4t + 2 \end{bmatrix}$$

We can calculate the length of the curve using the previous equation:

$$\begin{aligned} s(t) &= \int_0^t ||\Gamma'(u)||du \\ &= \int_0^t \sqrt{3^2 + 4^2}du = 5t \end{aligned}$$

We have that

$$s(t) = s = 5t \Longrightarrow t = \frac{s}{5} = t(s)$$

We just demonstrate with this example the relationship between the two parametrizations, which cal also be seen in the figure 1.3. The curve can be expressed also as:

$$\Gamma(s) = \begin{bmatrix} \frac{3}{5}s - 1 \\ \frac{4}{5}s + 2 \end{bmatrix}$$

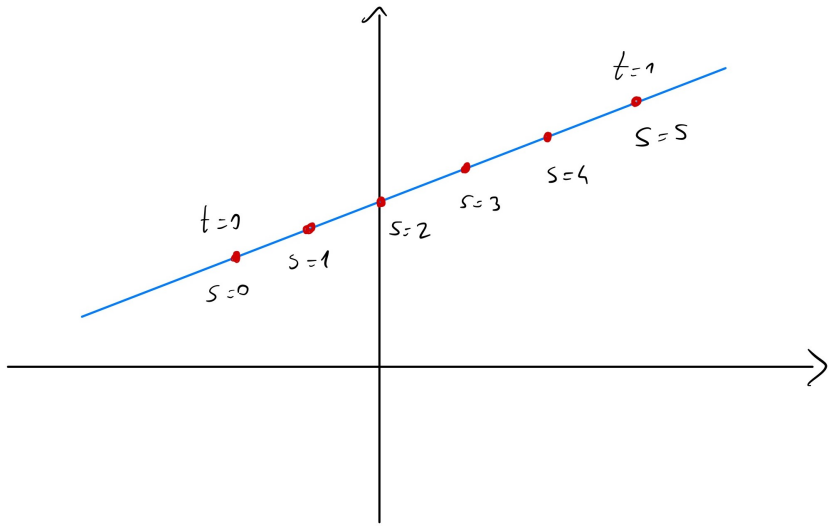

Figure 1.3: Example between the two parametrizations

Now let's define the *tangent vector* $\mathbf{T}$ and the *normal vector* $\mathbf{N}$ as:

$$\mathbf{T} = \frac{\Gamma'(t)}{||\Gamma'(t)||} \quad \mathbf{N} = \frac{\mathbf{T}'(t)}{||\mathbf{T}'(t)||} \tag{2}$$

where both vectors have norm $||\mathbf{T}|| = ||\mathbf{N}|| = 1$.
To make it more clear let's try to find the tangent and normal vector for a simple circumference of radius 1.

$$\Gamma(t) = \begin{bmatrix} cos(t) \\ sin(t) \end{bmatrix}$$

We have that

$$\mathbf{T}(\mathbf{t}) = \begin{bmatrix} -sin(t) \\ cos(t) \end{bmatrix} \quad \mathbf{N}(\mathbf{t}) = \begin{bmatrix} -cos(t) \\ -sin(t) \end{bmatrix}$$

In figure 1.4 we show how these vectors appear in two different points on the curve.

1.6 Curvature

Finally, we can define the *curvature* κ_r as:

$$\kappa_r(s_r) = ||\frac{d\mathbf{T}}{ds_r}|| = ||\mathbf{T}'(s_r)|| \tag{3}$$

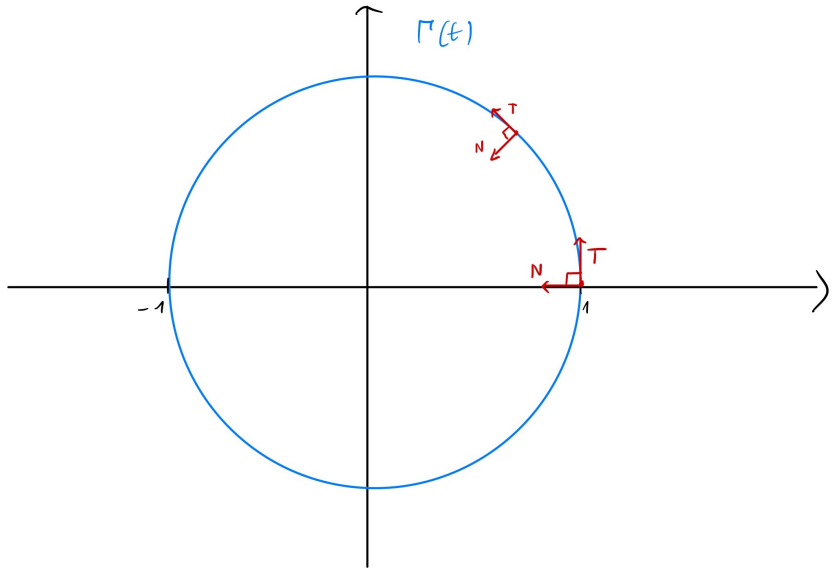

Figure 1.4: Example of tangent and normal vector in a simple circumference

In practice, the curvature approximate the best a curve in a given point with a circle of radius $r = \frac{1}{k_r}$.

This definition seems too complicated but in reality it simply measure how much the tangent vector change direction while moving. If κ_r=0, the curve is a straight road while if it's a high value it means that the curve is really tight.
An important relationship to highlight is the inverse of $s_r(t)$, defined as $t(s_r)$. Thus every point on the curve can be expressed as its arc-length parametrization.
The vectors describing the arc-length parametrization can be seen in figure 1.5.
In reality, it's really difficult to calculate the curvature using only the definition. There exists a theorem that give us a formula to calculate the curvature in each point of the curve if it is define in the plane $\mathbb{R}^2$:

$$\kappa_r(t) = \frac{|x'y'' - x''y'|}{((x')^2 + (y')^2)^{\frac{3}{2}}} \tag{4}$$

1.7 Arc-Length Parametrization - Implementation

To generate the data we need, let's first define the structure of the desired curve in the Cartesian coordinates (x, y). Then we can compute the derivatives for each spatial point of the curve (between each discretization step). Remember that we are working in a discrete environment, so:

$$\begin{cases} dx(i) = x(i+1) - x(i) \\ dy(i) = y(i+1) - y(i) \end{cases} \tag{5}$$

where $(x(i), y(i))$ and $(x(i+1)), y(i+1))$ are two consecutive spatial points on the curve Γ.

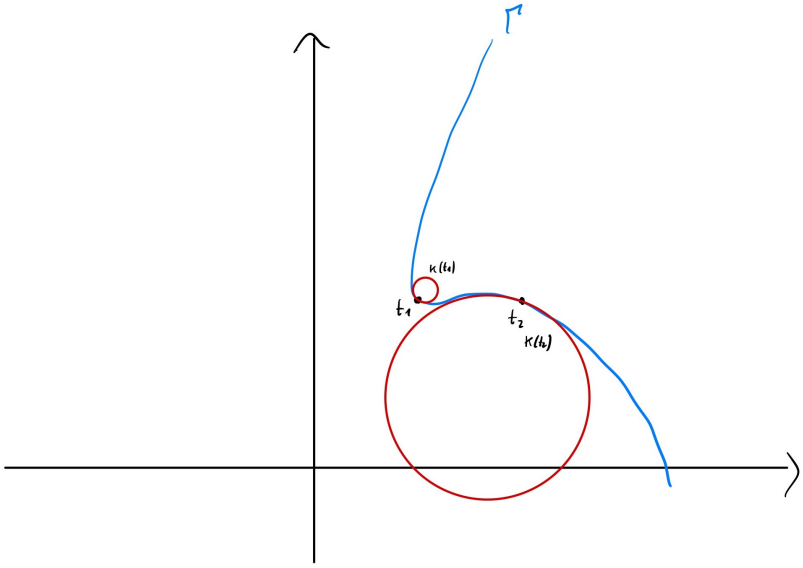

Figure 1.5: Application of the curvature in two points

Let's calculate also the second derivative:

$$\begin{cases} ddx(i) = dx(i+1) - dx(i) \\ ddy(i) = dy(i+1) - dy(i) \end{cases} \tag{6}$$

Now we can compute the length of each discretization step of the curve:

$$ds(i) = \sqrt{dx(i)^2 + dy(i)^2} \tag{7}$$

Let's compute the arc-length parametrization of the curve as following:

$$s_r(i) = \sum_{j=0}^{i} ds(j) \tag{8}$$

In the paper there is a figure (by reference 1.1) showing how it is possible to compute a circumference that approximates the arc-length considering the curvature $\kappa_r(s_r)$ in each discretization point. The curvature κ_r is computed as:

$$\kappa_r(i) = \frac{|dx\, ddy - ddx\, dy|}{(dx^2 + dy^2)^{\frac{3}{2}}} \tag{9}$$

The previous operations can be executed in MATLAB as shown in figures 1.6 and 1.7.

1.8 Velocity Profile

The authors of the paper recorded both trajectories and velocity profiles during the tests. Since velocities data are unavailable, we would like to reconstruct an appropriate profile. We just built the reference curve Γ that we will use as reference. As

```
% waypoints of the track
x_plot = 0:ds:40;
y_plot = zeros(1, length(x_plot));

% Transformation between points (x,y) to (k,theta)
% k = |x'y''-y'x''|/(x'^2+y'^2)^(3/2)

% dx=x' and dy=y' (differences at each step)
dx = diff(x_plot);
dy = diff(y_plot);
```

Figure 1.6: Definition of the path in Cartesian coordinates in MATLAB

```
ddx = diff(dx);
ddy = diff(dy);

num = dx(1:end-1).*ddy - dy(1:end-1).*ddx;
den = (dx(1:end-1).^2 + dy(1:end-1).^2).^(1.5);

kappa_r_vec = num ./ den;

% parametrization on the arc of the curves
ds_step = sqrt(dx.^2 + dy.^2); % this is the derivative
s_ref = [0, cumsum(ds_step)];

% computation of theta_r -> using [dy dy(end)] because diff() function
% provide the intervals which are N_points-1 wrt original points. We need
% to add the last piece as constat to have the same number of points
theta_r_vec = unwrap(atan2([dy dy(end)], [dx dx(end)]));
dtheta = diff(theta_r_vec);

% kappa_r = ||dtheta|| ==> normalize using ds
% other way: kappa_r_vec = dtheta./ds_step;
kappa_r_vec = [kappa_r_vec, kappa_r_vec(end), kappa_r_vec(end)]; % padding because of the differences
```

Figure 1.7: Definition of arc-length coordinates in MATLAB

velocity, for simplicity, we can just use a constant velocity.
In real-world applications, constant velocity is often not ideal. The curvature computed above is typically used to constrain the velocity: to maintain stability, the agent should slow down where the curvature is high (tight turns) and speed up where it is low.

2 Vehicle Prediction Model

To predict and optimize the vehicle trajectory, we first need to find a mathematical model that describes the behavior of the vehicle with respect to the reference curve Γ. The authors of the paper decide to consider a reference point on the car that coincides with the vehicle's rear axle center to derive the kinematic equations. It is preferable to select the time derivative of the curvature of Γ at a given point as input.
Classical approaches involve the formulation of the model with respect to a coordinate system; in our case, it is convenient to plan the future vehicle motion locally along a given reference curve Γ. The goal is to optimize the vehicle's lateral trajectory from the nominal trajectory Γ.
Let's introduce the following variables:

- d_r as the normal distance between the reference curve Γ and the vehicle's rear axle center position (x, y);
- θ_r as the reference curve Γ orientation at the correspondent base curve's point;
- κ_r as the reference curve Γ curvature at the correspondent base curve's point;
- s_r as the curve's arc length;
- θ as the vehicle's orientation;
- κ as the vehicle's trajectory curvature;

2.1 Non-Linear System's Equations

The non-linear system can be defined with the following equations:

$$\begin{cases} \dot{d}_r = v(t)\sin(\theta - \theta_r) \\ \dot{\theta} = v(t)\kappa \\ \dot{\kappa} = u \\ \dot{\theta}_r = \frac{v(t)\cos(\theta-\theta_r)}{1-\kappa_r d_r}\kappa_r \\ \dot{\kappa}_r = z \end{cases} \tag{10}$$

where $v(t)$ is the vehicle's velocity at each time instant, provided as a parameter by the reference trajectory, and z is an external signal derived from Γ.

2.2 Non-Linear System - Implementation

We will need a vehicle simulator to apply a control input and integrate the equations for each time instant using *ode45*. The simulator can be built as shown in figure 2.8.

```
function dxdt = solution(t, x, u, z, v)
    dxdt = zeros(5, 1);

    dxdt(1) = v*sin(x(2)-x(4));
    dxdt(2) = v*x(3);
    dxdt(3) = u;
    dxdt(4) = (v*cos(x(2)-x(4)))/(1-x(1)*x(5))*x(5);
    dxdt(5) = z;
end
```

Figure 2.8: Simulator of the vehicle in MATLAB

2.3 Linear System Model

The authors then make several assumptions to linearize the given non-linear system of equations. First of all, because the vehicle has to move closer to the reference trajectory, the difference in orientation $\theta - \theta_r$ is typically smaller than $20°$ at all times. This assumption allows us to state that

$$sin(\theta - \theta_r) \approx \theta - \theta_r$$

$$cos(\theta - \theta_r) \approx 1$$

Next, we can observe that d_r is typically small with respect to the reference curve's radius r_r which is defined as

$$r_r = \frac{1}{k_r}$$

So we can assume that

$$\frac{d_r}{r_r} = d_r \kappa_r << 1$$

such that the difference $1 - d_r \kappa_r$ is negligible.
We can now define the linear time-variant system model as

$$\dot{x}(t) = A_C(t)x(t) + B_C(t)u(t) + E_C(t)z(t) \tag{11}$$

where

$$A_C(t) = \begin{bmatrix} 0 & v(t) & 0 & -v(t) & 0 \\ 0 & 0 & v(t) & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & v(t) \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad B_C(t) = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \quad E_C(t) = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{bmatrix} \tag{12}$$

The state vector is defined as

$$x(t) = \begin{bmatrix} d_r \\ \theta \\ \kappa \\ \theta_r \\ \kappa_r \end{bmatrix} \quad \text{with} \quad x(0) = x_0 \tag{13}$$

For an efficient formulation of the system constraints, the authors defined a number of system output. Based on the idea of approximationg the vehicle's shape by several circles for fast collision checking, they locate three circle center points along the vehicle center line such that the vehicle shape is covered appropriately, as shown in figure 2.9.

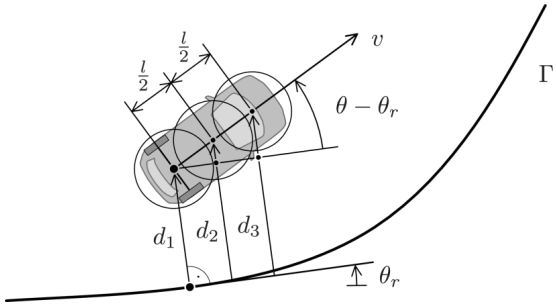

Figure 2.9: System model and outputs

The first and third circle center points are choose to coincide with the vehicle rear axle and front axle center, respectively. The second circle position is selected by distance $l/2$, in which l denotes the vehicle wheelbase. So, these distant values are:

- l_1 = 0
- l_2 = $\frac{l}{2}$
- l_3 = l

They define the circle's center positions with respect to the reference curve Γ as one component of the output system vector which is given by

$$d_i = d_r + l_i sin(\theta - \theta_r) \approx d + l_i(\theta - \theta_r), \quad i = 1, 2, 3 \tag{14}$$

These relation are showed in figure 2.9. The authors defined κ as an additional system output to account for physical limitations. The compact formulation of the output is described in equation

$$y(t) = C_C x(t) \tag{15}$$

$$C_C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 1 & \frac{l}{2} & 0 & -\frac{l}{2} & 0 \\ 1 & l & 0 & -l & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \tag{16}$$

The output vector is defined as

$$y(t) = \begin{bmatrix} d_1 \\ d_2 \\ d_3 \\ \kappa \end{bmatrix} \tag{17}$$

2.4 Discrete-Time Linear System Model

The authors described the transformation for the discrete-time model using the step-invariant transform

$$\Phi_k(T_s) = e^{A_C T_s} \xrightarrow{\mathcal{L}(\cdot)} [sI - A_C]^{-1} \tag{18}$$

This lead to the following discretized model:

$$A(k) = \Phi_k(T_s), \quad B(k) = \int_0^{T_s} \Phi_k(\tau) B_C \, d\tau, \quad E(k) = \int_0^{T_s} \Phi_k(\tau) E_C \, d\tau \tag{19}$$

Finally, we can write the discretized system's equation as

$$\begin{cases} x(k+1) = A(k)x(k) + B(k)u(k) + E(k)z(k) \\ y(k) = C(k)x(k) \end{cases} \quad \text{with } x(0) = x_0 \tag{20}$$

The system's disturbance $z(k)$ can be write as the first order hold of the difference of curvature at each time instant:

$$z(k) = \frac{\kappa_r(k+1) - \kappa_r(k)}{T_s} \tag{21}$$

2.5 Linear System Model - Implementation

The continuous-time model 12 can be implemented in MATLAB as shown in figure 2.10.

```
% Continuous-time model
Ac = zeros(n, n, N+1);
Bc = zeros(n, m, N+1);
Ec = zeros(n, o, N+1);
Cc = zeros(p, n, N+1);
v = v_pred;
for i = 1:N+1
    Ac(:,:,i) = [0      v(i)    0       -v(i)   0;
                 0      0       v(i)    0       0;
                 0      0       0       0       0;
                 0      0       0       0       v(i);
                 0      0       0       0       0];

    Bc(:,:,i) = [0;
                 0;
                 1;
                 0;
                 0];

    Ec(:,:,i) = [0;
                 0;
                 0;
                 0;
                 1];

    Cc(:,:,i) = [1      0       0       0       0;
                 1      1/2     0       -1/2    0;
                 1      1       0       -1      0;
                 0      0       1       0       0];
end
```

Figure 2.10: Contiuous-time model implementation in MATLAB

We can notice that the definition of time-varying matrices was achieved by defining three-dimensional matrices: the first two dimensions represent the system behaviour

with a fixed time-instant; the third dimension indicates the particular time-instant. I chose to use $N+1$ because the authors of the papers included $x(N)$ in the description forgetting $x(0)$, while I kept both.
Then the discrete-time model can be implemented using the *c2d* command in MATLAB, as shown in figure 2.11.

```
Ad = zeros(n, n, N+1);
Bd = zeros(n, m, N+1);
Ed = zeros(n, o, N+1);
Cd = zeros(p, n, N+1);
for i = 1:N+1
    A = Ac(:,:,i);
    B = Bc(:,:,i);
    C = Cc(:,:,i);
    E = Ec(:,:,i);

    sys_c = ss(A, B, C, 0);
    sys_d = c2d(sys_c, Ts);

    Ad(:,:,i) = sys_d.A;
    Bd(:,:,i) = sys_d.B;
    Cd(:,:,i) = sys_d.C;

    sys_c = ss(A, E, C, 0);
    sys_d = c2d(sys_c, Ts);

    Ed(:,:,i) = sys_d.B;
end
```

Figure 2.11: Discrete-time model implementation in MATLAB

Observe that I used the linearity of the system to separate the calculation of the B and E matrices because they are calculated in the same way but with a different input signal.

3 System Constraints

The generated trajectory is not isolated, when dealing with autonomous driving we need to take into account different objects as the static obstacles (for example the road lines) or dynamic traffic participants (for example other vehicles), as pictured in figure 3.12.

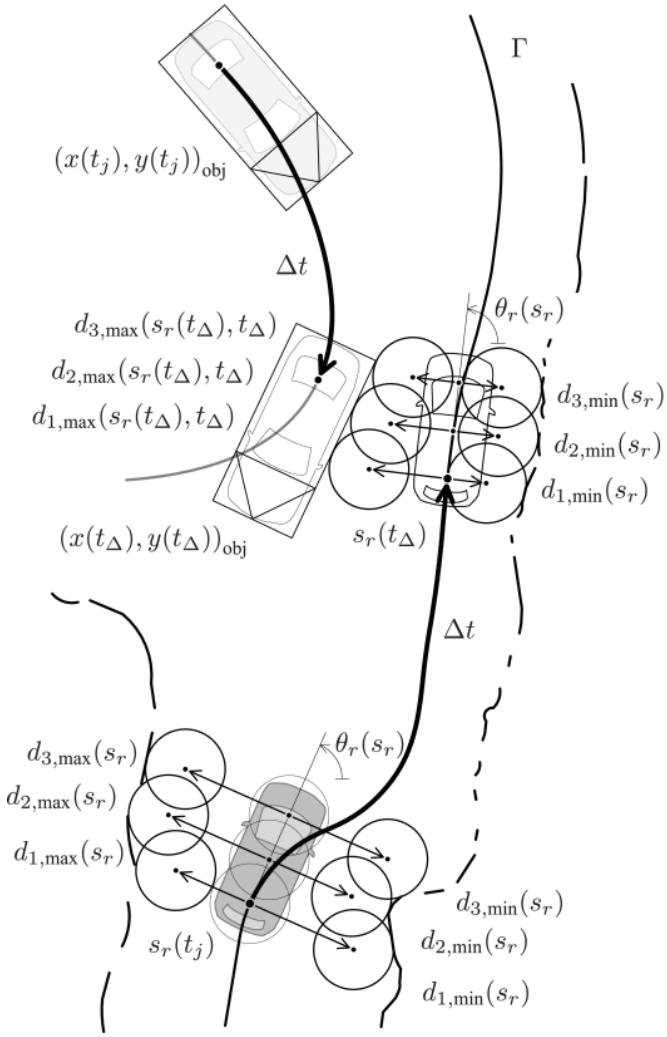

Figure 3.12: Static and dynamic obstacles

The authors, based on their idea of autonomous driving, decided to constrain the future vehicle motion such that collision with the obstacles (both static and dynamic) is avoided.
Given the output formulation that consider an approximation of the vehicle with three circles at distances d_i, $i = 1, 2, 3$, from the reference curve Γ, as shown in figure 2.9, the authors define the output constraints to limit the deviation from Γ. In figure 3.12 we can see that the circles are moved from both sides of the vehicle indicating their maximum (from left) and minimum (from right) values $d_{i,max}$, $d_{i,min}$ for two consecutive time instants t_j and t_Δ. In the second time instant, a moving object $(x(t_\Delta), y(t_\Delta))_{obj}$ is represented to show that the deviations d_i depends not

only on the reference curve but also on time t.
The system constraints must be formulated in a time-varying fashion such that

$$d_{i,min}(s_r(k), k) \leq d_i(k) \leq d_{i,max}(s_r(k), k), \quad i = 1, 2, 3 \tag{22}$$

considering the discrete-time system.
In addition, we need to constrain also the vehicle's lateral dynamics, such that kinematically feasible trajectories can be generated. The restriction arises from the physical limitations of the vehicle's steering actuator and wheel traction. The authors decided to constrain the curvature's rate of change by box contraint on the system input:

$$u_{min} \leq u(k) \leq u_{max} \tag{23}$$

Consequently, the vehicle's steering angle δ is also restricted, allowing for the definition of $\kappa_{min,\delta}$ and $\kappa_{max,\delta}$. Also, to account for limited wheel traction, we extend the limits to a time-varying parameter depending on the velocity profile $v(k)$ and the estimated friction parameter μ. The combination is expressed as:

$$max(\kappa_{min,\delta}, \kappa_{min,\mu}(k)) \leq \kappa(k) \leq min(\kappa_{max,\delta}, \kappa_{max,\mu}(k)) \tag{24}$$

$$\kappa_{min} \leq \kappa(k) \leq \kappa_{max} \tag{25}$$

The equations of $\kappa_{min,\mu} = \kappa_{min}(v(k), \mu)$ and $\kappa_{max,\mu} = \kappa_{max}(v(k), \mu)$ are derived using the *Kamm's circle*.

$$\kappa_{min,\mu} = -\frac{g\mu}{v^2} \quad \kappa_{max,\mu} = \frac{g\mu}{v^2} \tag{26}$$

3.1 System Constraints - Implementation

We can implement the system constraints by reducing the problem dimension to consider only the position of the rear axle center. In figure 3.13 we can see that the lateral constraints are calculated over a distance of $2 - r_{\text{circles}}$ because the road is $4m$ large but the circles have all a radius of $1m$ and that allow us to make the assumption only on the position of the rear axle center and not on the complete circumference.

```
N_curve = length(x_plot);

% Lateral limits because of the road
d_max_vec =  (2-r_circles)*ones(1, N_curve);
d_min_vec = (-2+r_circles)*ones(1, N_curve);

% Casual obstacle
obs_start = find(s_ref >= 7.5, 1);
obs_end   = find(s_ref >= 9, 1);

% Modification of the bounds to add the obstacle
d_min_vec(obs_start:obs_end) =d_min_vec(obs_start:obs_end) + 1.3;
```

Figure 3.13: Definition of the constraints and an obstacle in MATLAB

Also, it is possible to notice how an unplanned obstacle can be insert. It was not pre-computed in κ_r. Instead, it is added a-posteriori to test the robustness of the algorithm.

4 Cost Function

The objective is formulated as the minimization of $l(x(k), u(k))$, which is the quadratic running cost. The cost function chosen is inspired by a human driving behavior in which the driver attempt to follow the trajectory and minimize the vehicle's lateral acceleration and jerk. The applied input is designed to ensure a continuous and smooth steering action. The running cost is defined as:

$$l(x(k), u(k)) = w_d d_r^2 + w_\theta(\theta - \theta_r)^2 + w_\kappa \kappa^2 + w_u u^2 \tag{27}$$

with $w_d, w_\theta, w_\kappa, w_u > 0$. Let's analyze their meanings:

- w_d penalizes the lateral deviation d_r trying to steer it to 0 in order to be *on the line*;
- w_θ penalizes the difference between the vehicle's orientation θ and the reference curve angle of curvature θ_r because the objective is to make the vehicle follow perfectly the trajectory and this involve also the curvature angle. This weight is also important to prevent large deviations that will lead to worse performance of the linearized model;
- w_κ states that we need to steer to follow the curve but without excessive steering. It is preferrable to minimize the steering effort;
- w_u is the penalization on the input because considering the importance of the *human-like* steering decision, rapid changes in direction significantly reduce passenger comfort.

The authors decided also to parametrize these parameters on the desired future velocity, such that

$$w_{x_i}(v(k)), \quad x_i = d, \theta, \kappa, u$$

For example, we may choose to increase w_κ as the speed increase while reducing w_d. The cost function can now be expressed as follows

$$l(x(k), u(k)) = x(k)^T Q(k) x(k) + u(k)^T R(k) u(k) \tag{28}$$

where

$$Q(k) = \begin{bmatrix} w_d(k) & 0 & 0 & 0 & 0 \\ 0 & w_\theta(k) & 0 & -w_\theta(k) & 0 \\ 0 & 0 & w_\kappa(k) & 0 & 0 \\ 0 & -w_\theta(k) & 0 & w_\theta(k) & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix} \quad \text{and} \quad R(k) = [w_u(k)] \tag{29}$$

with $Q(k)$ being positive-semidefinite and $R(k)$ positive-definite matrices, to satisfy MPC assumptions.

4.1 Cost Function - Implementation

In order to implement the $Q(k)$ and $R(k)$ matrices we first define the dimensions, as shown in figure 4.14. We can see that these cost matrices consider the state and input values $x(k)$ and $u(k)$ from $k = 0$ to $k = N - 1$. We also defined the costs as constant apart from w_d and w_κ which are defined as

$$w_d(k) = \frac{2}{v(k)} \qquad w_\kappa(k) = \frac{v(k)}{10}$$

such that when the first decreases, the second increases as a function of the velocity $v(k)$, as stated before.

```
% Optimal control problem - cost function
w_theta = 10;
w_u = 10;

Q = zeros(n, n, N);
R = zeros(1, N);
for i = 1:N+1
    w_d = 2/v_pred(i);
    w_k = v_pred(i)/10;
    Q(:,:,i) = [w_d          0            0          0            0;
                0            w_theta      0          -w_theta     0;
                0            0            w_k        0            0;
                0            -w_theta     0          w_theta      0;
                0            0            0          0            0];

    R(i) = w_u;
end
```

Figure 4.14: Implementation of the time-varying cost function in MATLAB

5 Constrained Optimal Control Problem

Given the definition of the cost function, a constrained linear quadratic optimal control problem is formulated over a prediction horizon N with $k = 0, \ldots, N$. To solve it using quadratic optimizer, we need to use a formulation to consider every time instant. Tha authors decided to use a sparse formulation in a batch approach to solve the optimization problem for N steps directly. So, for each simulation time instant t_j the optimizer compute N steps forward control input and only the first one is applied. Let's define the augmented state and input vector:

$$X = \begin{bmatrix} x(k|k) \\ x(k+1|k) \\ x(k+2|k) \\ \vdots \\ x(k+N-1|k) \\ x(k+N|k) \end{bmatrix} \in \mathbb{R}^{n(N+1)} \quad U = \begin{bmatrix} u(k|k) \\ u(k+1|k) \\ u(k+2|k) \\ \vdots \\ u(k+N-1|k) \end{bmatrix} \in \mathbb{R}^{N} \tag{30}$$

where $n = 5$ is the state-space dimension and $m = 1$ the input dimension. We can also define the disturbance vector:

$$Z = \begin{bmatrix} z(k) \\ z(k+1) \\ z(k+2) \\ \vdots \\ z(k+N-1) \end{bmatrix} \in \mathbb{R}^{N} \tag{31}$$

and the output vector:

$$Y = \begin{bmatrix} y(k|k) \\ y(k+1|k) \\ y(k+2|k) \\ \vdots \\ y(k+N-1|k) \\ y(k+N|k) \end{bmatrix} \in \mathbb{R}^{p(N+1)} \tag{32}$$

where $p = 4$ is the number of outputs.
The linearized equations can now be expressed in a dense formulation:

$$\begin{cases} X = \mathcal{A}x(k|k) + \mathcal{B}U + \mathcal{E}Z \\ Y = \mathcal{C}X \end{cases} \tag{33}$$

where

$$\mathcal{A} = \begin{bmatrix} I \\ A(k) \\ A(k+1)A(k) \\ \vdots \\ \prod_{q=1}^{N-1} A(k+N-q) \end{bmatrix} \tag{34}$$

$$\mathcal{B} = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ B(k) & 0 & 0 & \dots & 0 \\ A(k+1)B(k) & B(k+1) & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ (\prod_{q=1}^{N-1} A(k+N-q))B(k) & \dots & \dots & B(k+N-1) & 0 \end{bmatrix} \tag{35}$$

$$\mathcal{E} = \begin{bmatrix} 0 & 0 & 0 & \dots & 0 \\ E(k) & 0 & 0 & \dots & 0 \\ A(k+1)E(k) & E(k+1) & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ (\prod_{q=1}^{N-1} A(k+N-q))E(k) & \dots & \dots & E(k+N-1) & 0 \end{bmatrix} \tag{36}$$

$$\mathcal{C} = \begin{bmatrix} C(k) & 0 & 0 & \dots & 0 \\ 0 & C(k+1) & 0 & \dots & 0 \\ \vdots & \vdots & \dots & \vdots & \dots \\ 0 & 0 & 0 & \dots & C(k+N-1) \end{bmatrix} \tag{37}$$

Equivalently, the running cost function can be summed up for all k in the entire prediction horizon N (assume to start at 0 for semplicity):

$$J(X,U) = x^T(N)Px(N) + \sum_{k=1}^{N-1} x(k)^T Q(k)x(k) + u(k)^T R(k)u(k) \tag{38}$$

$$= X^T \mathcal{Q} X + U^T \mathcal{R} U \tag{39}$$

where

$$\mathcal{Q} = \begin{bmatrix} Q(0) & 0 & 0 & \dots & 0 & 0 \\ 0 & Q(1) & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & Q(N-1) & 0 \\ 0 & 0 & 0 & \dots & 0 & P \end{bmatrix} \tag{40}$$

and

$$\mathcal{R} = \begin{bmatrix} R(0) & 0 & 0 & \dots & 0 \\ 0 & R(1) & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 0 & 0 & 0 & \dots & R(N-1) \end{bmatrix} \tag{41}$$

Stability in the closed loop system is difficult to prove because of the not full controllability (actually, even not stabilizability) of the system and the poles of the A's matrices on the unitary-circle, which cannot prove us anything because of the linearization. A practical approach to give some margin of *stability* is to choose P matrix with bigger weights than the Q's matrices.
As we want to solve the optimization problem for the input vector sequence U, let's substitute the dynamics in the cost function:

$$J(x_0, Z, U) = X^T \mathcal{Q} X + U^T \mathcal{R} U \tag{42}$$

$$= U^T \mathcal{H} U + 2(x_0^T \mathcal{F} + Z^T \mathcal{G})U + \mathcal{O} \tag{43}$$

where

$$\mathcal{H} = \mathcal{B}^T \mathcal{Q} \mathcal{B} + \mathcal{R} \quad \mathcal{F} = \mathcal{A}^T \mathcal{Q} \mathcal{B} \quad \mathcal{G} = \mathcal{E}^T \mathcal{Q} \mathcal{B} \tag{44}$$

with $J(x_0, Z, U)$ being only dependent on a variable U and $\mathcal{O}$ representing a constant additional cost. To prepare a suitable formulation for the system constraints the authors defined the output sequence Y:

$$Y = \mathcal{C}[\mathcal{A}x_0 + \mathcal{B}U + \mathcal{E}Z] \tag{45}$$

5.1 Implementation in MATLAB

When implementing in MATLAB we can do this:

```
A_big = zeros(n*(N+1), n);
A_big(1:n,:) = eye(n);
A_prod = Ad(:,:,1);
for i = 1:N
    idx_n = i*n;
    A_big(idx_n+1:idx_n+n,:) = A_prod;
    A_prod = Ad(:,:,i+1)*A_prod;
end


B_big = zeros(n*(N+1),m*N);
for j = 1:N
    B_prod = Bd(:,:,j);
    k = (j-1)*m;
    for i = j:N
        idx_n = i*n;
        B_big(idx_n+1:idx_n+n,k+1:k+m) = B_prod;
        B_prod = Ad(:,:,i+1) * B_prod;
    end
end
```

Figure 5.15: Implementation of $\mathcal{A}$ and $\mathcal{B}$

5.2 Slack Variables

In order to avoid infeasibility issues, the authors decided to use a technique to soft the constraints. They introduced the *slacked variables* $\epsilon \geq 0$. By including these variables into the optimization problem as an additional optimization variable, we can penalize the violation of constraints in the cost function.
In this paper, the authors select a smaller portion of outputs $y_{i_s}(k), \forall i_s \in \mathcal{I}_s \subset [1, \ldots, p]$ to soften at all time steps $k \in \mathcal{K}_s \subset [1, \ldots, N]$. The motivation behind this choice is due to the sensibility of the sensors which may detect the same object in two different position at consecutive time instants because of the noise, even if the

```
C_big = zeros(p*(N+1), n*(N+1));
for i = 1:N+1
    idx_n = (i-1)*n;
    idx_p = (i-1)*p;
    C_big(idx_p+1 : idx_p+p, idx_n+1 : idx_n+n) = Cd(:,:,i);
end

Eps_big = zeros(n*(N+1),o*N);
for j = 1:N
    Eps_prod = Ed(:,:,j);
    k = (j-1)*o;
    for i = j:N
        idx_n = i*n;
        Eps_big(idx_n+1:idx_n+n,k+1:k+o) = Eps_prod;
        Eps_prod = Ad(:,:,i+1)*Eps_prod;
    end
end
```

Figure 5.16: Implementation of $\mathcal{C}$ and $\mathcal{E}$

```
% Now we insert x(N) on the Q-matrix as P
w_dP = 1000;
w_thetaP = 1000;
w_kP = 1000;
P =    [w_dP          0              0          0              0;
        0             w_thetaP       0          -w_thetaP      0;
        0             0              w_kP       0              0;
        0             -w_thetaP      0          w_thetaP       0;
        0             0              0          0              0];
```

Figure 5.17: Implementation of P

object is static. As this situation affects the feasibility of the problem only shortly before an obstacle is passed, the authors soften the collision avoidance constraints $y_{i_s}, \forall i_s \in \mathcal{I}_s = [1, 2, 3]$ over a short prediction horizion N_s with $N_s << N$.
We can define a total of $r = 2$ slack variable, one per vehicle's side.

$$\epsilon = \begin{bmatrix} \epsilon_u \\ \epsilon_l \end{bmatrix} \tag{46}$$

where l stands for the lower side of the vehicle with respect to the curve and u as the upper side. The total number of soft outputs equations is $p_s = |\mathcal{I}_s| N_s$ in which $pN = p_s + p_h$.
Now we can finally modify the cost function to include the slack variables:

$$\tilde{J}(x_0, Z, U, \epsilon) = J(x_0, Z, U) + k_1^T \epsilon + \epsilon^T K_2 \epsilon \tag{47}$$

$$= \tilde{U}^T \tilde{H} \tilde{U} + \tilde{F} \tilde{U} \tag{48}$$

with

$$k_1 = \begin{bmatrix} k_{11} \\ k_{22} \end{bmatrix} \quad K_2 = \begin{bmatrix} k_{21} & 0 \\ 0 & k_{22} \end{bmatrix} \tag{49}$$

```
Q_big = zeros(n*(N+1), n*(N+1));
for i = 1:N
    idx_n = (i-1)*n;
    Q_big(idx_n+1:idx_n+n,idx_n+1:idx_n+n) = Q(:,:,i);
end
idx_n = N*n;
Q_big(idx_n+1:idx_n+n,idx_n+1:idx_n+n) = P;

R_big = zeros(m*N, m*N);
for i = 1:N
    idx_m = (i-1)*m;
    R_big(idx_m+1:idx_m+m,idx_m+1:idx_m+m) = R(i);
end
```

Figure 5.18: Implementation of $\mathcal{Q}$ and $\mathcal{R}$

```
H_big = B_big'*Q_big*B_big + R_big;
```

```
F_big = A_big'*Q_big*B_big;
G_big = Eps_big'*Q_big*B_big;

f = 2*(x0'*F_big+z'*G_big);
f = f';
```

Figure 5.19: Implementation of $\mathcal{H}$ and f

and

$$\tilde{H} = \begin{bmatrix} H & 0 \\ 0 & K_2 \end{bmatrix} \quad \tilde{F} = 2(x_0^T F + Z^T G)k_1^T \quad \tilde{U} = \begin{bmatrix} U \\ \epsilon \end{bmatrix} \tag{50}$$

We can also modify the vector sequence of the output for separating the soft outputs Y_s and the hard outputs Y_h:

$$Y_h = \mathcal{C}_h[\mathcal{A}x_0 + \mathcal{B}U + \mathcal{E}Z] \tag{51}$$

$$Y_s = \mathcal{C}_s[\mathcal{A}x_0 + \mathcal{B}U + \mathcal{E}Z] \tag{52}$$

where

$$\mathcal{C}_h = \begin{bmatrix} C_{h,1} & 0 & 0 & \dots & 0 \\ 0 & C_{h,2} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & 0 \\ 0 & 0 & 0 & \dots & C_{h,N} \end{bmatrix} \quad \mathcal{C}_s = \begin{bmatrix} C_{s,1} & 0 & 0 & \dots & 0 \\ 0 & C_{s,2} & 0 & \dots & 0 \\ \vdots & \vdots & \vdots & \dots & 0 \\ 0 & 0 & 0 & \dots & C_{s,Ns} \end{bmatrix} \tag{53}$$

Let's define now $C_{h,k}$ and $C_{s,k}$:

$$C_{h,k} = \begin{cases} \begin{bmatrix} 0 & 0 & 0 & 1 \end{bmatrix} C_k & \forall k \in \mathcal{K}_s \\ C_k & \forall k \notin \mathcal{K}_s \end{cases} \tag{54}$$

$$C_{s,k} = I_{|\mathcal{I}_s| \times p} C_k \quad \forall k \in \mathcal{K}_s \tag{55}$$

Now we can build the constraints for the soft and hard output:

$$Y_{h,min/max} = \begin{bmatrix} Y_{h,1,min/max} \\ Y_{h,2,min/max} \\ \vdots \\ Y_{h,N,min/max} \end{bmatrix} \qquad Y_{s,min/max} = \begin{bmatrix} Y_{s,1,min/max} \\ Y_{s,2,min/max} \\ \vdots \\ Y_{s,N_s,min/max} \end{bmatrix} \tag{56}$$

and then

$$Y_{h,k,min/max} = \begin{cases} \begin{bmatrix} \kappa_{min/max}(k) \end{bmatrix} & \forall k \in \mathcal{K}_s \\ \begin{bmatrix} d_{1,min/max}(k) \\ d_{2,min/max}(k) \\ d_{3,min/max}(k) \\ \kappa_{min/max}(k) \end{bmatrix} & \forall k \notin \mathcal{K}_s \end{cases} \tag{57}$$

$$Y_{s,k,min/max} = \begin{bmatrix} d_{1,min/max}(k) \\ d_{2,min/max}(k) \\ d_{3,min/max}(k) \end{bmatrix} \qquad \forall k \notin \mathcal{K}_s \tag{58}$$

Finally we are able to write the inequality constraints for the modified cost function as:

$$Y_h - Y_{h,max} \leq 0, \quad Y_s - Y_{s,max} \leq 0, \quad -\epsilon_u \leq 0 \tag{59}$$

$$-Y_h + Y_{h,min} \leq 0, \quad -Y_s + Y_{s,min} \leq 0, \quad -\epsilon_l \leq 0 \tag{60}$$

The authors then integrate all the inequality constraints in a big matrix to follow this inequality equation:

$$\tilde{A}_c \tilde{u} \leq \tilde{b}_c \tag{61}$$

where

$$\tilde{A}_c = \begin{bmatrix} \begin{bmatrix} I_N \\ -I_N \end{bmatrix} & 0_{2N \times r} \\ \begin{bmatrix} \mathcal{C}_h \mathcal{B} \\ -\mathcal{C}_h \mathcal{B} \end{bmatrix} & 0_{2p_h \times r} \\ \begin{bmatrix} \mathcal{C}_s \mathcal{B} \\ -\mathcal{C}_s \mathcal{B} \end{bmatrix} & \begin{bmatrix} -1_{p_s} & 0_{p_s} \\ 0_{p_s} & -1_{p_s} \end{bmatrix} \\ 0_{r \times N} & -I_r \end{bmatrix} \tag{62}$$

and

$$\tilde{b}_c = \begin{bmatrix} \begin{bmatrix} \dot{\kappa}_{max} 1_N \\ -\dot{\kappa}_{min} 1_N \end{bmatrix} \\ \begin{bmatrix} Y_{h,max} - \mathcal{C}_h[\mathcal{A}x_0 + \mathcal{E}Z] \\ -Y_{h,min} + \mathcal{C}_h[\mathcal{A}x_0 + \mathcal{E}Z] \end{bmatrix} \\ \begin{bmatrix} Y_{s,max} - \mathcal{C}_s[\mathcal{A}x_0 + \mathcal{E}Z] \\ -Y_{s,min} + \mathcal{C}_s[\mathcal{A}x_0 + \mathcal{E}Z] \end{bmatrix} \\ 0_r \end{bmatrix} \tag{63}$$

5.3 Slack Variables and Constraints - Implementation

First of all we can implement the slack variables as shown in figure 5.20. The costs are three order bigger than the values of the $Q(k)$ matrices because we will penalize a lot if the constraints are violated.

```
% Softer problem
Ns = 5;
k_1 = [1e3;
       1e3];

K_2 = diag([1e5; 1e5]);

% now u_tilde = [u', [eps_u, eps_l]']'
p_s = 3;
p_h = p;

H_tilde = blkdiag(H_big, K_2);
F_tilde = [f; k_1];
```

Figure 5.20: Implementation of the slack variables in the cost function

Then we can modify the $\mathcal{C}$ matrix to include the soft constraints and divide them from the hard ones, which allow us to define $\mathcal{C}_h$ and $\mathcal{C}_s$, as shown in figure 5.21.

```
Ch_big = [];
C_h_Ks = zeros(1, n, Ns);
for i = 1:Ns
    C_h_Ks(:,:,i) = [0, 0, 0, 1]*Cd(:,:,i);
    Ch_big = blkdiag(Ch_big, C_h_Ks(:,:,i));
end
C_h = zeros(p, n, N+1-Ns);
for i = 1:N+1-Ns
    C_h(:,:,i) = Cd(:,:,i+Ns);
    Ch_big = blkdiag(Ch_big, C_h(:,:,i));
end

C_s = zeros(p_s, n, Ns);
Cs_big = [];
for i = 1:Ns
    C_s(:,:,i) = [1, 0, 0, 0; 0, 1, 0, 0; 0, 0, 1, 0]*Cd(:,:,i);
    Cs_big = blkdiag(Cs_big, C_s(:,:,i));
end
```

Figure 5.21: Implementation of $\mathcal{C}_h$ and $\mathcal{C}_s$

In chapter 3, we discussed about the choice of the steering constraints which can be determined by selecting the closer bounds between the physical steering system limits and the wheel steering, due to the friction coefficient and the velocity profile. We can implement this choice as follow in figure 5.22.

```
% Constraints for the output
k_max = zeros(N+1, 1);
k_min = zeros(N+1, 1);

for i = 1:N+1
    v = v_pred(i);

    % Kamm circle for the tire maximum grip that influences the max
    % steering action
    k_max_mu = (mu*g)/(v^2);

    % close the bounds around the minimum and maximum steering actions
    k_max(i) = min(k_max_delta, k_max_mu);
    k_min(i) = max(k_min_delta, -k_max_mu);
end

k_min_history(time_instant) = k_min(1);
k_max_history(time_instant) = k_max(1);
```

Figure 5.22: Implementation of the steering constraints in MATLAB

Now we can finally implement the output constraints by combining the first part of the soft constraints and the second part that involves the hard constraints, as shown in figure 5.23.

```
s = 3;
y_s_max = zeros(s*Ns, 1); y_s_min = zeros(s*Ns, 1);
for i = 1:Ns
    is = (i-1)*s;
    y_s_max(is+1:is+s) = [d_max_fut(i); d_max_fut(i); d_max_fut(i)];
    y_s_min(is+1:is+s) = [d_min_fut(i); d_min_fut(i); d_min_fut(i)];
end

y_h_max_Ks = k_max(1:Ns);
y_h_min_Ks = k_min(1:Ns);

y_h_max_k = zeros(p*(N+1-Ns), 1);
y_h_min_k = zeros(p*(N+1-Ns), 1);
for i = 1:(N+1-Ns)
    ip = (i-1)*p;
    y_h_max_k(ip+1 : ip+p) = [d_max_fut(Ns+i); d_max_fut(Ns+i); d_max_fut(Ns+i); k_max(Ns+i)];
    y_h_min_k(ip+1 : ip+p) = [d_min_fut(Ns+i); d_min_fut(Ns+i); d_min_fut(Ns+i); k_min(Ns+i)];
end

y_h_max = [y_h_max_Ks; y_h_max_k];
y_h_min = [y_h_min_Ks; y_h_min_k];
```

Figure 5.23: Implementation of the output constraints in MATLAB

Now we can assemble all the parts relatives to the constraints and build the inequality matrices $\tilde{A}_c$ and $\tilde{b}_c$. The construction of the first one is shown in figure 5.24. To simplify the code I write it as the union of more basic blocks M_{ij}.
In the same way we can define $\tilde{b}_c$ as shown in figure 5.25.
Finally, after creating all the matrices relative to the cost function and to the inequality constraints, we can use *quadprog()* in MATLAB to solve iteratively the optimization problem. The application of *quadprog()* is shown in figure 5.26.

```
ps = 3*Ns;
ph = p*(N+1)-ps;
r = 2;

Cs_big = [Cs_big, zeros(ps, n*(N+1-Ns))];

M_11 = [eye(N); -eye(N)];
M_21 = [Ch_big*B_big; -Ch_big*B_big];
M_12 = zeros(2*(N), r);
M_22 = zeros(2*ph, r);
M_31 = [Cs_big*B_big; -Cs_big*B_big];
M_41 = zeros(r, N);
M_32 = [-ones(ps, 1), zeros(ps, 1); zeros(ps, 1), -ones(ps, 1)];
M_42 = -eye(r);

Ac_tilde = [M_11, M_12;
            M_21, M_22;
            M_31, M_32;
            M_41, M_42];
```

Figure 5.24: Implementation of $\tilde{A}_c$

```
V_1 = [u_max*ones(N, 1);
       -u_min*ones(N,1)];

V_2 = [y_h_max-Ch_big*(A_big*x0+Eps_big*z);
       -y_h_min+Ch_big*(A_big*x0+Eps_big*z)];

V_3 = [y_s_max-Cs_big*(A_big*x0+Eps_big*z);
       -y_s_min+Cs_big*(A_big*x0+Eps_big*z)];

V_4 = zeros(r, 1);

bc_tilde = [V_1; V_2; V_3; V_4];
```

Figure 5.25: Implementation of $\tilde{b}_c$

```
% I recall the function quadprog
% UMPC contains the optimal input sequence
% (H+H')/2 symmetrized a matrix in case of small numerical errors
[UMPC,FVAL,EXITFLAG]=quadprog((H_tilde+H_tilde')/2,F_tilde,Ac_tilde,bc_tilde,[],[],[],[],[]);

% exitflag tells what was the result of the optimization
% if -2 the problem was not feasible
if EXITFLAG==-2
    disp('Something wrong with the optimization')
    time_instant
    keyboard
else
    u_curr=UMPC(1);
    u=[u u_curr];
    v_curr = v(time_instant);
    z_curr = z(1);

    [~, x_sim] = ode45(@(t, x) solution(t, x, u_curr, z_curr, v_curr), [time_instant*Ts, (time_instant+1)*Ts], x(:, end));

    xnext = x_sim(end, :)';
    x=[x xnext];
    x0 = x(:, end);
end
```

Figure 5.26: How to use quadprog()

6 Results

6.1 Paper's results

The authors tested the code on three different environments.
First of all, we can see a parking situation in figure 6.27. Using some advanced sensors in real environment, they measure the obstacles and show in the simulation, the optimization of the lateral distance from the pre-computed trajectory.

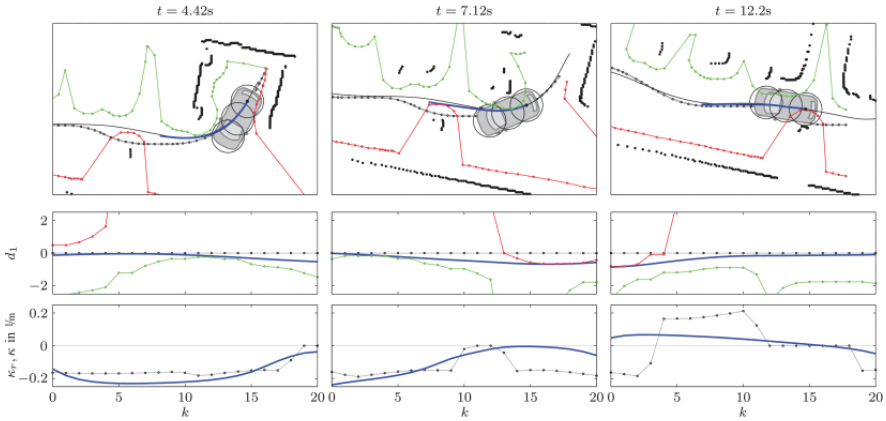

Figure 6.27: Simulation of a parking exit

Then, they recorded a trajectory and velocity profile over a roundabout. As you can see in figure 6.28, the vehicle try to cut the corner in the most efficient way to avoid losing velocity.
The last simulation shows a simple curve where a dynamic obstacle, represented as another moving vehicle, tries to cut the corner of our vehicle. In figure 6.29 we clearly see the change of the trajectory to avoid the collision.

6.2 My results

The complete code is attached to this report and it is called *lateral_trajectory_MPC.m*. After running it, the script presents six different test cases, as shown in figure 6.30.

6.2.1 Straight road

The first study case is a simply straight road with an unplanned obstacle (it wasn't pre-computed, it simulates, for example, some road works). I show the results from the *bird's eye view* in figure 6.31 and the time analysis in figure 6.32.
The vehicle chooses to steer a bit in the obstacle's direction to curve enough to pass it. Here the velocity profile is at its maximum of $8\frac{m}{s}$.

6.2.2 Sine() road

Because of the difficulty on road generation due to building continuous and smooth curves that allows a feasible steering, I chose to rely on classical function from calcu-

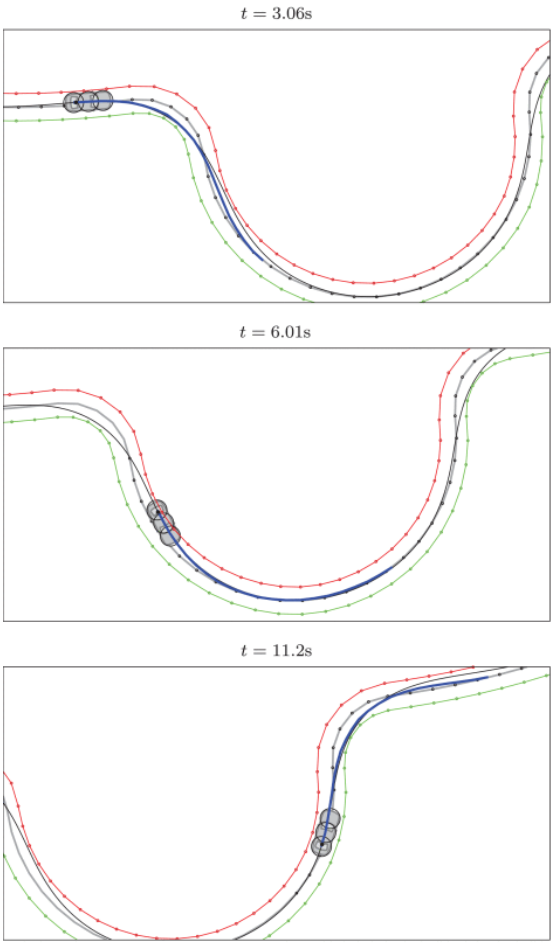

Figure 6.28: Simulation of a roundabout

lus. In this case, I used a sine wave with amplitude of 20 and wave-length $\lambda = 80m$. The results are shown in figures 6.33 and 6.34.
It is easy here to see the optimization in taking the curves: the vehicle tries to cut the corner, where it's possible, to exit from the curves with the maximum velocity of $20\frac{m}{s}$.

6.2.3 Roundabout

The third test case try to simulate the roundabout environment used in the paper. I couldn't find any good approximation using *spline()* interpolation so, I decided to create a simpler road profile by using a Gaussian function. In particular, I set $\mu = 30$ and $\sigma = 10$ with an amplitude parameter $\alpha = 10$. You can see the results in figures 6.35 and 6.36.
Even here it is possible to see the optimization of the trajectory to gain the maximum velocity in the end of the track, where the vehicle exit the roundabout.

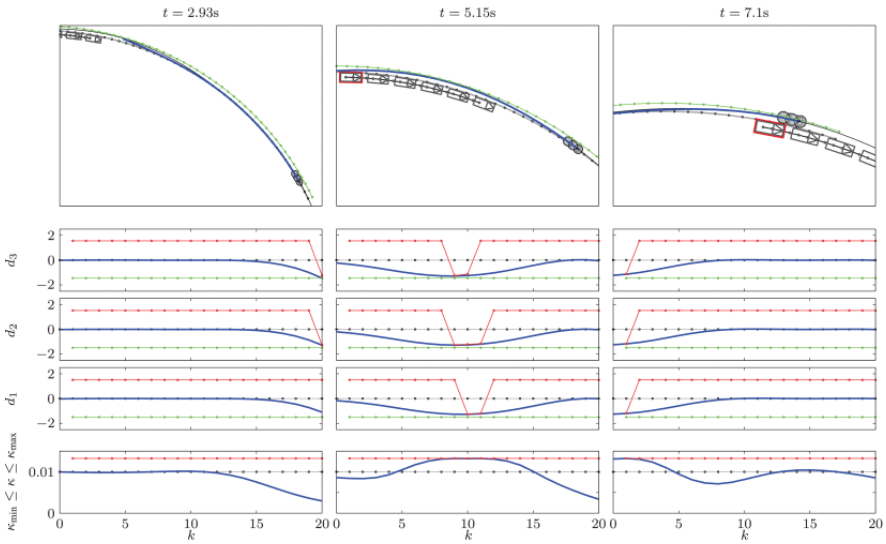

Figure 6.29: Simulation of a curve with a dynamic obstacles

```
-----------------------------------------------------------------
      LATERAL VEHICLE TRAJECTORY OPTIMIZATION USING LMPC
-----------------------------------------------------------------
Select the test you want to execute:
  1: Straight road with an obstacle
  2: Sine() road without obstacles
  3: Roundabout
  4: Parking scenario with more obstacles
  5: Lemniscate
  6: Simple curve
-----------------------------------------------------------------
Type the number of the test you want to execute: |
```

Figure 6.30: Menu of the main code

6.2.4 Parking scenario

The fourth environment I tried to simulate was the parking one. The road structure is simple and it relies on the conjunction of two straight road and a quarter of circumference to simulate the curve in the end of the parking spot. The velocities are low and I placed different obstacles to simulate a really bad environment. The results are shown in figures 6.37 and 6.38.
As in the first scenario, the obstacles weren't planned in the road limits but where sensed and avoided during the driving simulation.

6.2.5 Lemniscate

The fifth simulation involved a particular curve called *lemniscate*. The parametric equations describing the curve are:

$$\Gamma(t) = \begin{bmatrix} \frac{a\,cos(t)}{1+sin^2(t)} \\ \frac{a\,sin(t)\,cos(t)}{1+sin^2(t)} \end{bmatrix} \tag{64}$$

I decided to set $a = 60$ and the result can be seen in figures 6.39 and 6.40.

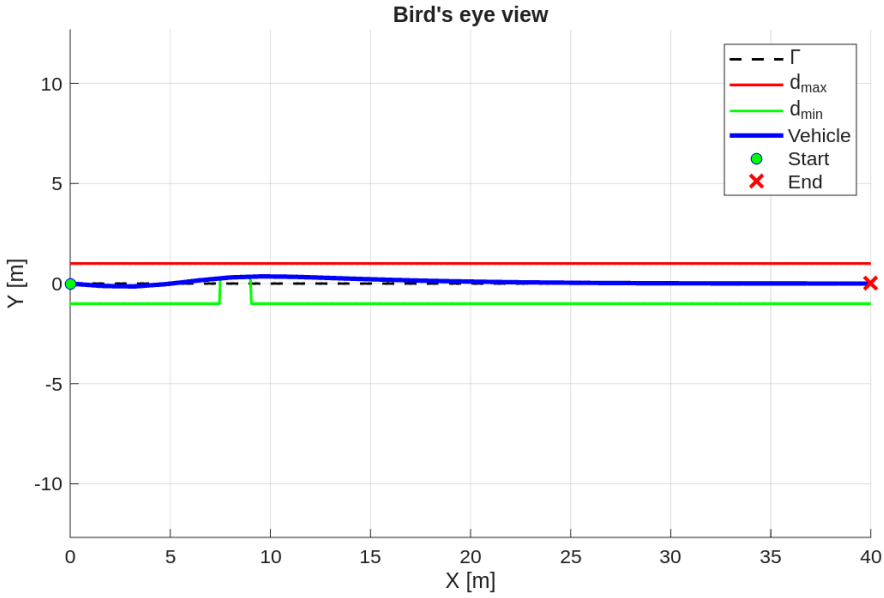

Figure 6.31: First simulation: straight road - Bird's eye view

6.2.6 Simple Curve

The last environment is a simple curve calculated on a forth of circumference of radius of 60m. The results are shown in figures 6.41 and 6.42.

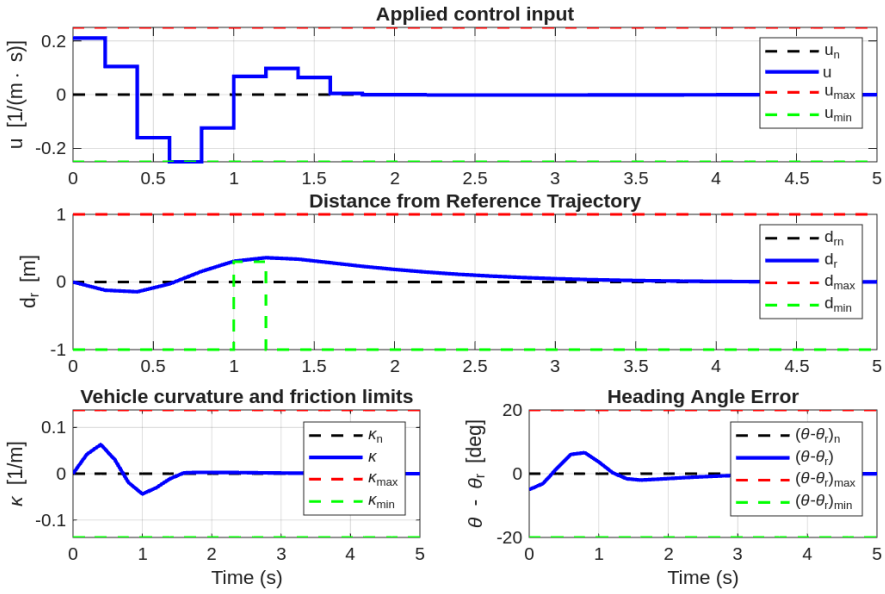

Figure 6.32: First simulation: straight road - Time analysis

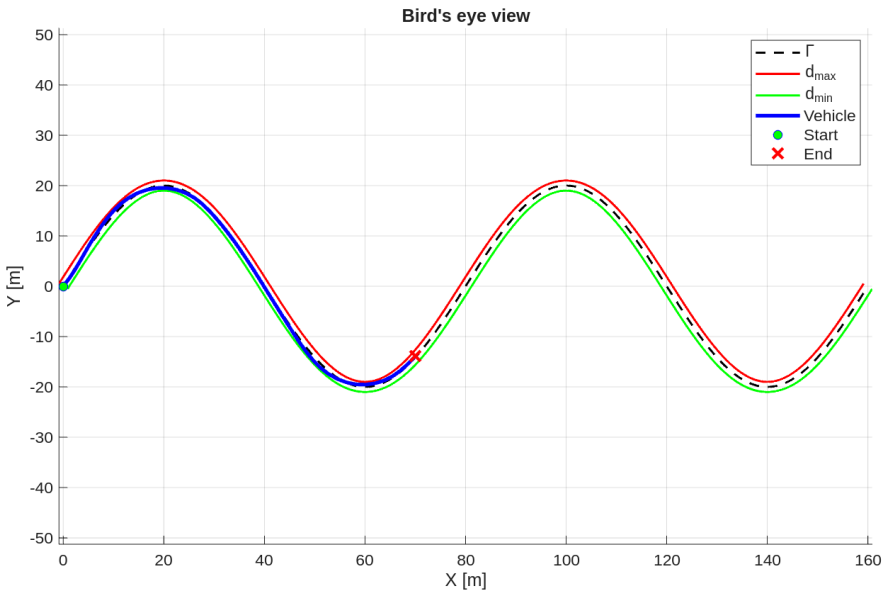

Figure 6.33: Second simulation: sine() road - Bird's eye view

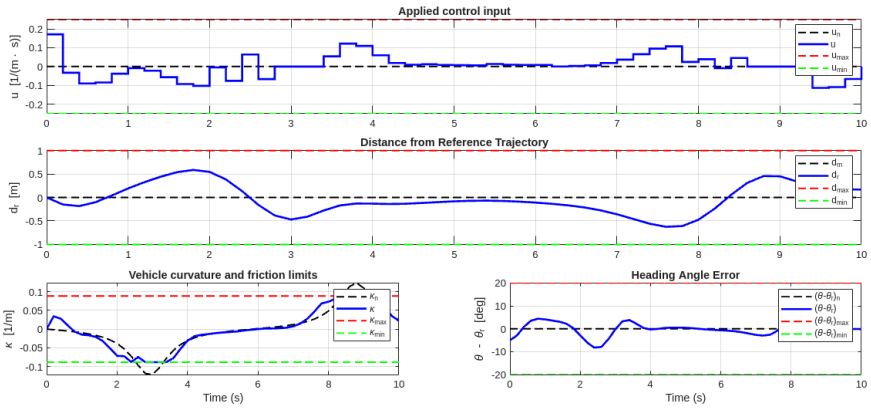

Figure 6.34: Second simulation: sine() road - Time analysis

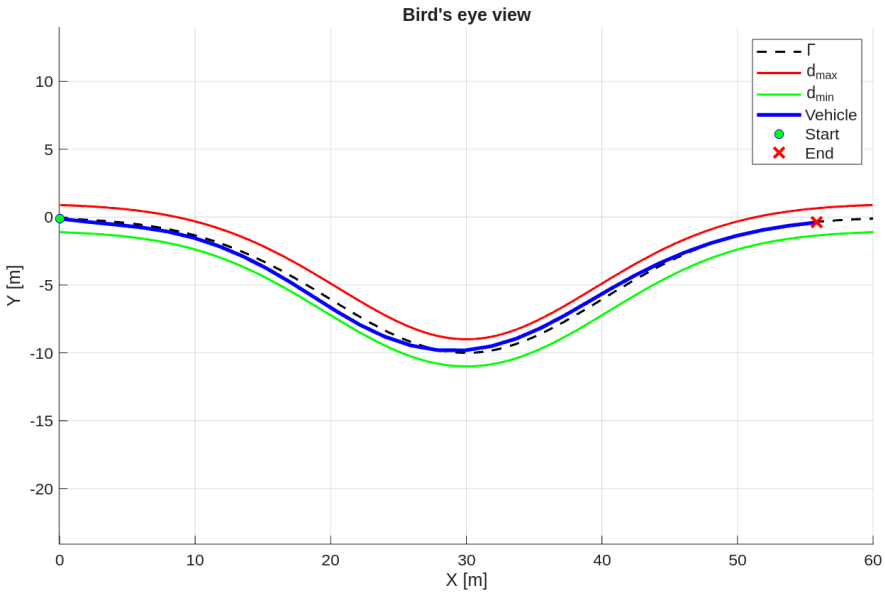

Figure 6.35: Third simulation: roundabout - Bird's eye view

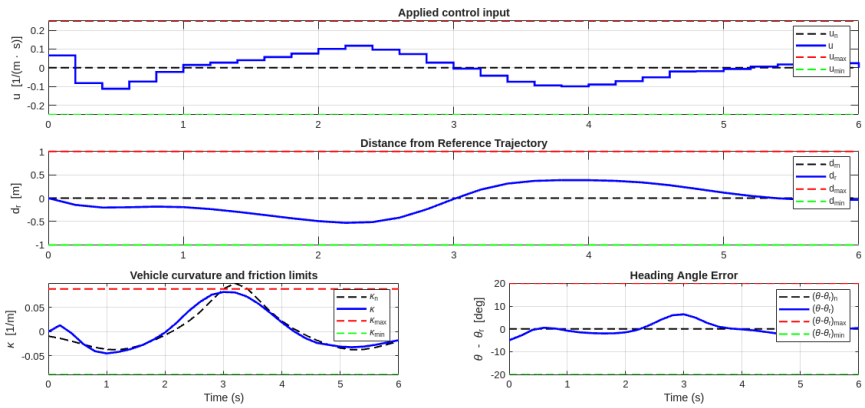

Figure 6.36: Third simulation: roundabout - Time analysis

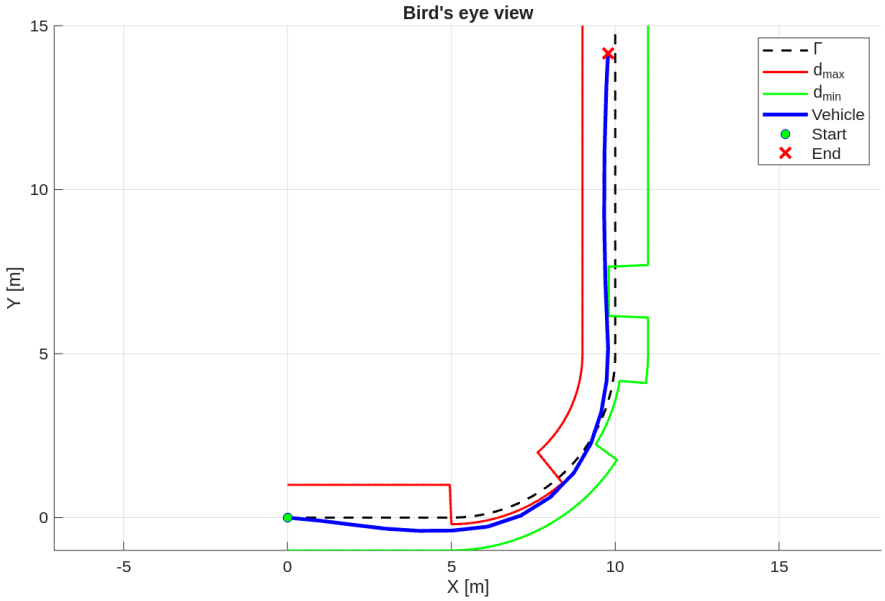

Figure 6.37: Fourth simulation: parking - Bird's eye view

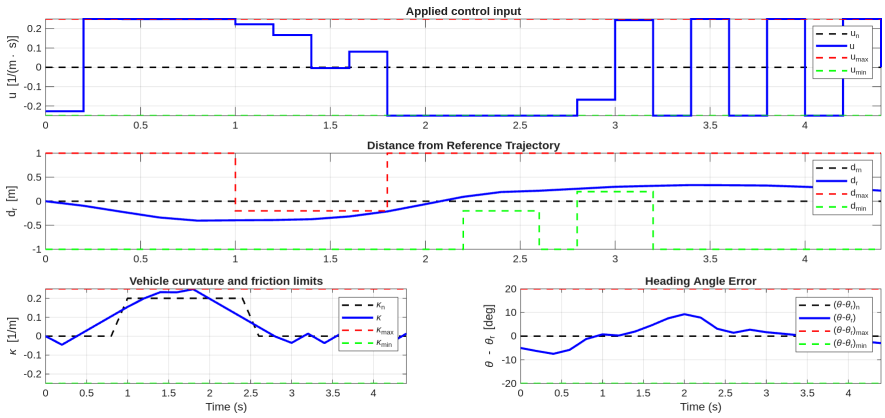

Figure 6.38: Fourth simulation: parking - Time analysis

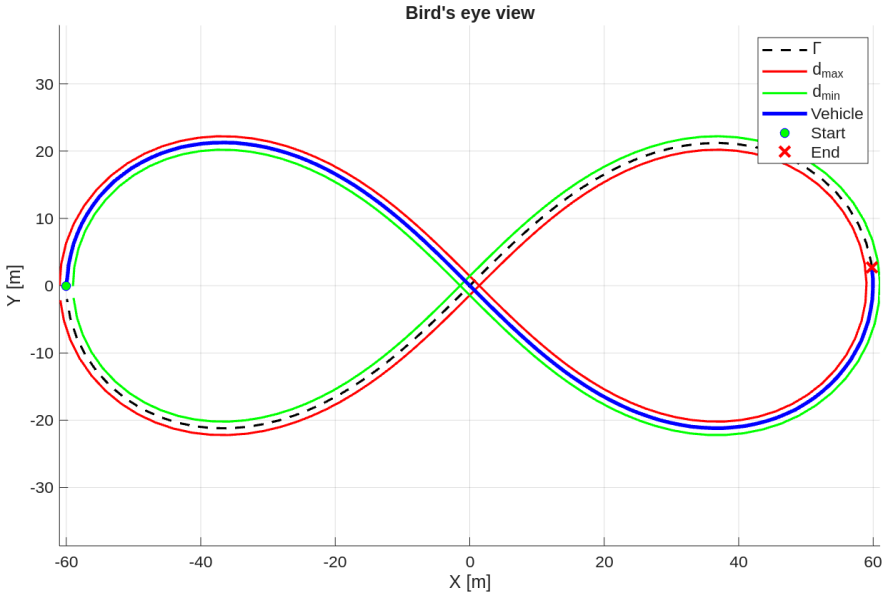

Figure 6.39: Fifth simulation: lemniscate - Bird's eye view

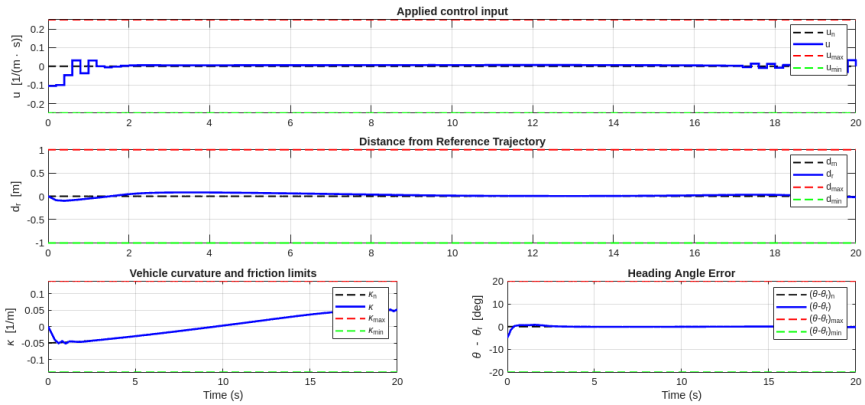

Figure 6.40: Fifth simulation: lemniscate - Time analysis

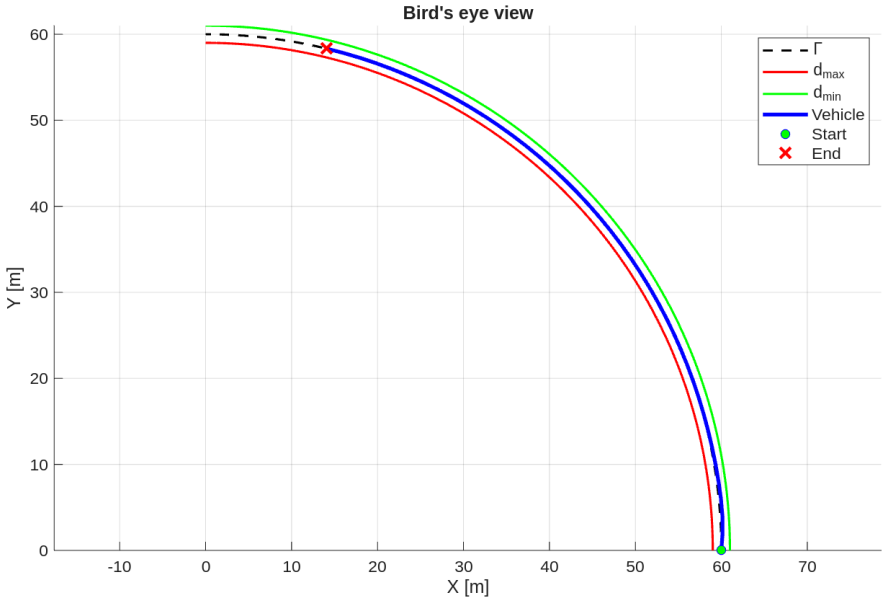

Figure 6.41: Sixth simulation: simple curve - Bird's eye view

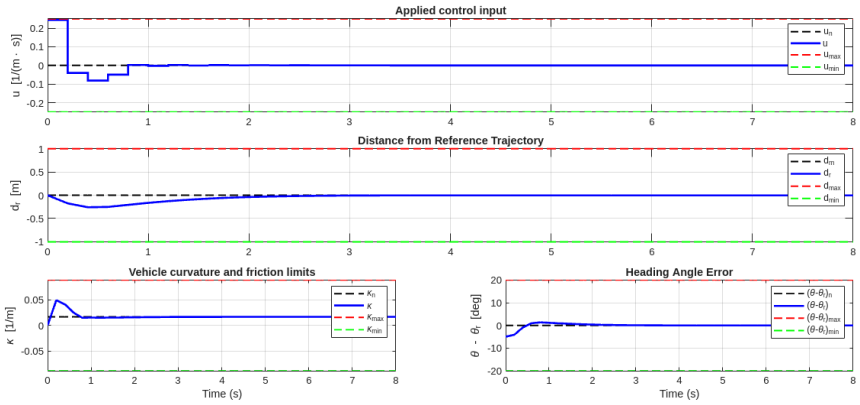

Figure 6.42: Sixth simulation: simple curve - Time analysis

Conclusion

The goal of this project was to study and understand well the paper in order to implement it in MATLAB. We could demonstrates that optimal control is effective in complex, real-world systems with such variability and even noise. Even if the authors used a real physical setup such as two BMW vehicles, it was really interesting to study and apply some differential geometry concepts, like arc-length parametrization, to create different trajectories for different tracks. The main difficulty here involved the feasibility of the bounds and the curves with respect to the real world. While the authors had the confidence about their trajectories to work in real life, I could not prove it. In fact, in several occasions I was unable to generate more complex trajectories with some interpolation function like spline, because they were not able to guarantee some good margin of confidence. Many times the optimizer did not find any feasible points during the curves, maybe it involved also numerical errors. In the end the best way to produce differential curves was to use differential function or the combination of them.
The aim of the work was also to show that it is possible to control very well a non-linear system by applying MPC on the linearized one. The only problem involves the time-varying property of the MPC because of the linearization in each time-instant position of the vehicle. In fact, this is easy to solve in any computer but could be expensive in computational terms in an on-board pc. Authors showed that this is not actual the case because they managed to apply the algorithm to a real vehicle by writing the program in C and running it on a 1GHz processor.
The use of the slack-variables was game changing to address feasibility issues and manage to solve the *false detection* of obstacle made by the sensors during rapid curves or by noise.
Different from the classical study-case, the stability of the MPC was not formally proven. Because of the complexity of the system and the fact that is preferrable to ensure feasibility, the authors decided to ensure a degree of stability by using a long horizon and penalizing a lot the last term with P. Proving stability is difficult because the system is not fully controllable, nor stabilizable because of the state space representation. So, the technique we study to ensure stability are not used in this paper and they are really hard to apply in the case of a linearized time-varying system.
In the end, the results on the four different cases work and show really well how the vehicle try to take the curves in the most efficient way (by cutting corner) and also in the most safe way (by avoiding the unplanned obstacle).
The implementation was quite hard to develop but for sure this paper was really interesting to study and it would be really challenging to apply this knowledge to a real vehicle.